

Instrucciones

- Lea con detenimiento y desarrolle **individualmente** cada una de las actividades a realizar durante la experiencia.
- Cree un archivo comprimido con extensión **.zip**, **.rar** o **.tar.gz** que incluya todos los archivos con extensión **.cpp** y **.h** utilizados para el desarrollo. El nombre del archivo debe tener el siguiente formato: **TEL102_C3_Nombre_Apellido.zip** (Ej. TEL102_C3_Patricio_Olivares.zip), sin incluir tildes.
- Las evaluaciones de la asignatura deben ser realizadas de manera **presencial** según lo estipulado en las reglas de clases y evaluaciones de la asignatura.
- Enviar el archivo a través de la página de aula del ramo, sección “Control 3” hasta las 12:15:00 del día Martes 29/10/2024 hora local continental de Chile.
- Utilice herramientas conocidas o aprendidas en clases. **No copie literalmente de recursos online.**
- La evaluación es de carácter individual. Está **estrictamente prohibido** tanto la comunicación entre estudiantes, así como el uso de herramientas generativas tales como chatGPT, Github copilot, y similares.
- Sea riguroso con las instrucciones de desarrollo.
- ¡Éxito!

1. AutoLogic - Gestión de Flota de Vehículos

La compañía de transporte **AutoLogic** desea implementar un sistema para gestionar vehículos inteligentes en su flota. Cada vehículo puede ser de distintos tipos, tales como *Vehículo a Bencina*, *Vehículo Eléctrico*, y *Vehículo Híbrido*, y debe registrarse con datos específicos. Para ello, han solicitado sus servicios como programador. Su tarea es crear un programa utilizando el paradigma de **Programación Orientada a Objetos (POO)** en C++. El programa debe permitir tanto el registro como la gestión eficiente de sus vehículos. Recuerde que, al aplicar POO, la definición de la clase debe ir en un archivo **.h**, mientras que la implementación de sus métodos debe realizarse en un archivo **.cpp**.

Para el desarrollo, debe utilizar como base los siguientes archivos disponibles en la plataforma Aula de la asignatura:

- `main.cpp`
- `funciones.h`

Estos archivos contienen la lógica principal del programa y algunas funciones definidas. El código que escriba **debe ser compatible** y funcionar correctamente en conjunto con el código entregado.

1. (30pts) Defina e implemente la clase abstracta **Vehiculo**, que debe contener los siguientes **atributos protegidos**:

- **Marca del vehículo:** Almacena la marca del vehículo.
- **Modelo del vehículo:** Almacena el modelo del vehículo.
- **Año de fabricación:** Indica el año en que se fabricó el vehículo.

Además, la clase debe incluir:

- Un **constructor** para inicializar los atributos mencionados.
- Un método virtual puro `virtual void mostrarDetalles() = 0;` para hacer la clase abstracta.

Puede agregar otros métodos y funcionalidades que considere útiles o necesarios para el correcto funcionamiento de la clase.

2. (35pts) Defina e implemente las clases derivadas **VehiculoBencina**, **VehiculoElectrico**, y **VehiculoHibrido**, las cuales heredan de la clase **Vehiculo**. Las clases deben cumplir con los siguientes requisitos:

- **VehiculoAutonomo** debe incluir un atributo adicional para la capacidad del estanque de combustible.
- **VehiculoElectrico** debe incluir un atributo adicional para la capacidad de la batería (en kWh).
- **VehiculoHibrido** debe tener atributos para la capacidad de la batería y el tipo de combustible.
- Cada clase debe sobrescribir el método `mostrarDetalles()` para mostrar todos los atributos, incluidos los heredados de **Vehiculo**.

Puede agregar otros métodos y funcionalidades que considere útiles o necesarios para el correcto funcionamiento de la clase.

3. (35pts) Implemente las funciones definidas en el archivo `funciones.h` en su correspondiente archivo `funciones.cpp`:

- Función `void registrarVehiculo(std::vector<Vehiculo*>& flota)` para registrar nuevos vehículos.
- Función `void mostrarFlota(const std::vector<Vehiculo*>& flota)` para mostrar los detalles de todos los vehículos en la flota.
- Función `void liberarFlota(std::vector<Vehiculo*>& flota)` para liberar la memoria de todos los vehículos antes de finalizar el programa.

Puede agregar otros métodos y funcionalidades que considere útiles o necesarios para el correcto funcionamiento de la clase.